

ACE: An Agentic Manipulation Harness for Semantic Task Composition with a Reusable Visuomotor Policy

Anonymous Authors

Abstract—General-purpose manipulation requires both semantic reasoning over task constraints and reliable execution of contact-rich actions. We present ACE, an agentic manipulation harness that composes a high-level language agent with a reusable mask-conditioned visuomotor policy. Given an open-ended instruction, the agent solves semantic constraints, binds objects to destination roles, and decomposes the task into executable transfers represented by tracked pick-and-place masks. Execution feedback supports outcome assessment, re-grounding, and retry, while persistent object and task context preserves earlier associations when manipulation changes visible cues. We evaluate ACE on two physical multi-step tabletop tasks, Semantic Formula Assembly and Constraint Retrieval. The visuomotor policy is trained only on generic pick-and-place demonstrations and reused without complete demonstrations of either evaluation task, enabling task-level zero-shot composition. Across 20 randomized trials per task, ACE achieves 70% and 80% success, respectively, compared with 55% and 70% without persistent context. These results suggest that an agentic harness can extend a primitive-trained manipulation policy to semantically distinct tasks through explicit object-destination interfaces and closed-loop execution feedback.

I. INTRODUCTION

Natural language provides an intuitive interface for robotic control, allowing users to specify goals at the task level. Executing these goals requires connecting semantic constraints to physical actions [1], [2], [3]. Instructions may define relationships that several objects must jointly satisfy, alongside the spatial roles those objects must occupy. The robot must resolve these relationships, execute the corresponding transfers, and assess the resulting arrangement.

In Semantic Formula Assembly, the agent selects number and operator cubes that jointly satisfy a requested arithmetic relation, assigns them to operand and operator slots, and checks the expression realized by their placement. Constraint Retrieval maps selection conditions to eligible objects and their destinations. These tasks share a manipulation primitive while requiring different semantic decisions. Formula assembly specifically couples constraint solving, role-to-location binding, and verification of a relation expressed by several physical objects.

Learned manipulation policies provide reusable physical skills [4], [5], and language-model agents can compose perception and control calls while incorporating execution feedback [6], [7]. Each semantic decision must identify both an object and its destination, and their image regions must be updated as execution changes the scene. Manipulation can also alter the evidence available for checking completion: a cube may turn so that its identifying pattern is no longer visible. This motivates retaining both current spatial targets and earlier object context.

We present ACE (Agentic Control for Embodied manipulation), a framework that connects semantic workflows to learned pick-and-place execution through **unified, tracked object-and-destination masks**. Given an instruction, the agent generates semantic sub-goals and invokes two robot-facing skills: `MaskInterface` grounds the active sub-goal, and `PickPlace` executes the corresponding transfer. A single mask assigns distinct pixel values to the pick object and place region. Tracking updates these regions during execution, and the resulting mask conditions a reusable vision-action policy.

The object–destination assignment connects these task requirements to execution: semantic roles specify the intended transfer, and the tracked mask supplies its current image regions. ACE targets *task-level zero-shot composition*. The same controller, trained on generic pick-and-place demonstrations, executes both tasks without full-task demonstrations of either evaluation task. The agent composes object selection, destination assignment, and sequencing at inference (Fig. 1).

ACE separates live workflow and tracking state from persistent object and conversational context. Current masks specify where to act; retained context makes earlier object associations and task information available when assessing the resulting scene. After an attempt, the agent checks the observation and execution report against the active sub-goal, then advances or refreshes grounding. We evaluate this workflow on two tasks and ablate persistent context while keeping the DP controller and live state fixed.

Our contributions are:

- A framework connecting task-level constraint solving, object–destination role binding, and learned pick-and-place execution through a shared controller.
- An execution interface combining tracked pick/place regions with persistent object and task context for grounding and outcome assessment as manipulation changes the scene.
- A physical evaluation achieving 70% and 80% success on Semantic Formula Assembly and Constraint Retrieval, with matched visual-input and persistent-context ablations and qualitative analysis of completion uncertainty after cube rotation.

II. RELATED WORK

A. Language-Model Agents for Manipulation

Large language models (LLMs) and vision-language models (VLMs) support robotic task understanding and embodied

Different task constraints, one learned pick-and-place policy

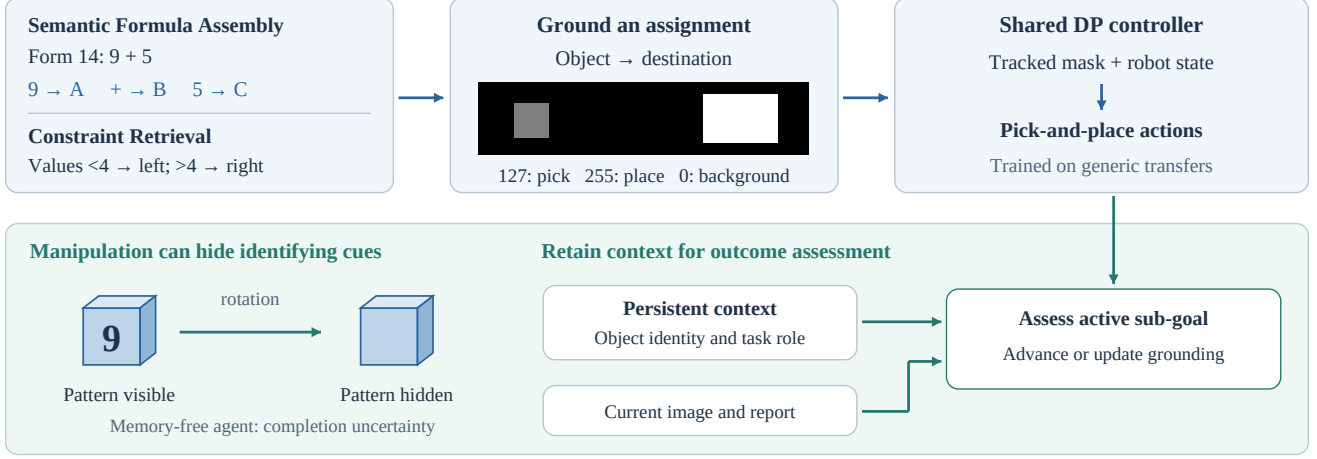


Fig. 1. **Semantic task composition in ACE.** Different task constraints produce object–destination assignments that share a tracked mask interface and a learned pick-and-place policy. Persistent context retains earlier object and task information for assessing outcomes when manipulation changes visible identifying cues. Masks and cubes are schematic; the full system is shown in Fig. 2.

planning [2], [7], [3], [8], including instruction parsing, task decomposition, and tool use [9], [10], [11]. Code as Policies composes perception and control APIs through language-model-generated programs [6]. Inner Monologue incorporates environment feedback into task planning [7]. ACE builds on these agent functions and focuses on their connection to a learned pick-and-place primitive: each workflow step supplies an explicit object–destination assignment through a tracked visual mask.

B. Visual Grounding and Mask-Conditioned Control

Visual grounding and tracking connect task descriptions to targets in changing scenes [12], [13], [14]. MOO [15] encodes object locations using distinct pixel values and supplies them with RGB observations and a skill instruction to a learned policy. ACE uses tracked object and destination regions to execute semantic workflows with a controller conditioned on masks and proprioception. A closely related system, DexGraspVLA [16], combines a VLM planner, SAM/Cutie target masks, and a diffusion-based grasping controller. Its planner monitors grasp outcomes and sequences multiple grasping steps; a successful grasp triggers a scripted placing motion in the described pipeline. ACE encodes both the pick object and the place destination in one tracked mask, conditioning a learned policy for the complete pick-and-place transfer. This interface maps semantic placement roles, such as operand and operator slots, to the shared controller.

VoxPoser grounds language into three-dimensional value maps for model-based motion planning [17]. ACE uses a two-dimensional pick-and-place mask as the input to a learned manipulation policy. Object selection and destination assignment are resolved by the agent, while the controller maps the tracked spatial targets and robot state to actions.

C. Task-Level Semantic Requirements

Task requirements provide a further distinction from these closely related systems. MOO evaluates known manipulation skill templates with novel object descriptions [15]. DexGraspVLA’s long-horizon evaluation includes clearing a scene and selecting objects by color or category for sequential grasping [16]. ACE’s formula task requires selecting operands and an operator that jointly satisfy an arithmetic relation, binding them to distinct expression slots, and assessing the assembled expression. This links a constraint over several selected objects to role-specific physical placement. Constraint Retrieval shares the condition-based selection structure of such evaluations and assigns selected objects to instruction-specified destinations. Both tasks use the same primitive-trained controller, while ACE composes the workflow at inference.

D. Persistent Context for Sequential Decisions

Agents use object histories, scene information, and conversational context to retain information across decisions [18], [19], [20]. ACE separates live workflow and tracking state from object and conversational context retained across agent calls. The persistent context is available when physical actions change the visual cues used for outcome assessment. The memory ablation evaluates this context jointly while keeping the DP controller unchanged.

III. METHOD

We consider tabletop manipulation from a live image stream $I_1, I_2, \dots$ and a natural-language instruction u . ACE composes task-level semantic reasoning with a reusable Vision-Action (VA) policy. Let $\mathcal{D}_{\text{prim}}$ denote the generic mask-conditioned pick-and-place demonstrations used to train this policy, and $\mathcal{T}_{\text{eval}}$ the semantic evaluation tasks. *Task-level zero-shot composition* means that the policy is

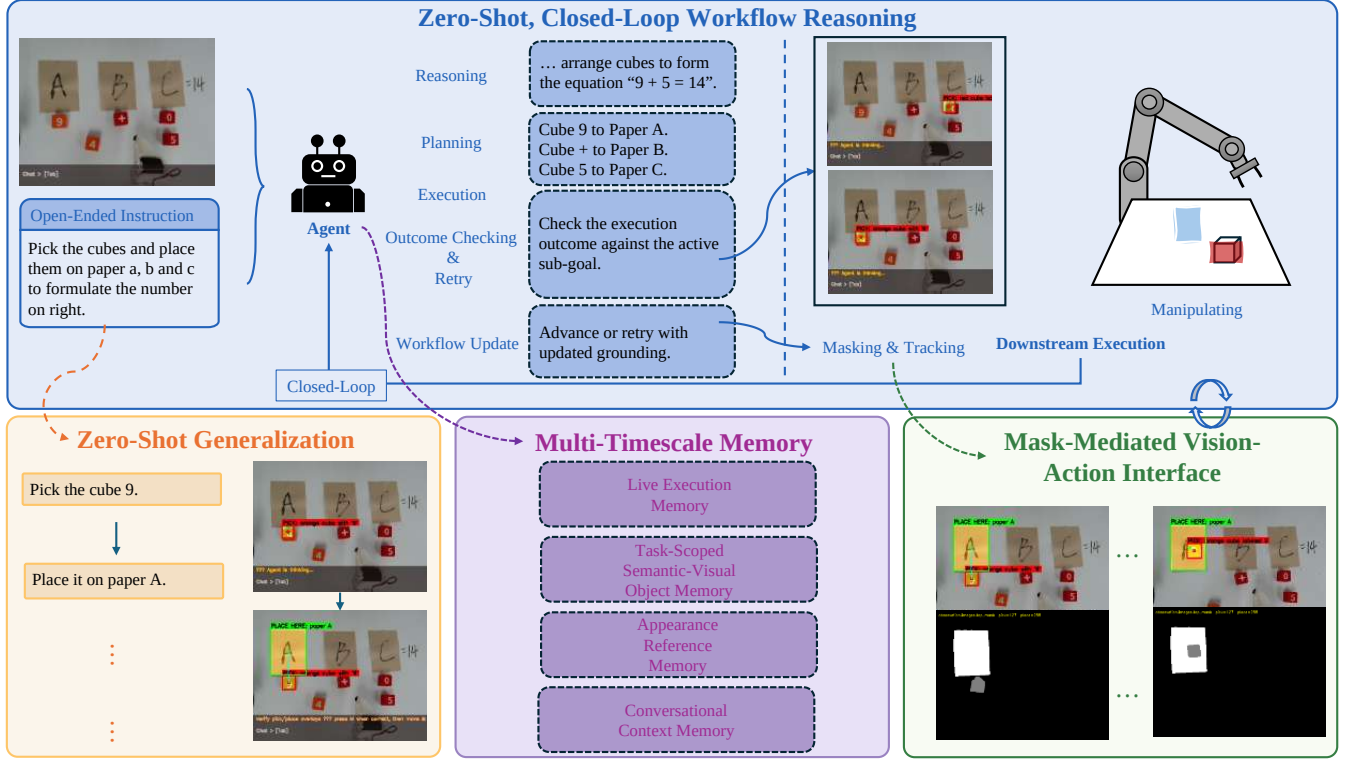


Fig. 2. **Overview of ACE.** The agent decomposes an instruction into semantic sub-goals, grounds each into a tracked pick-and-place mask, and invokes a reusable VA policy. The scene-state manager retains live execution state and persistent object and task context. Post-execution feedback guides workflow progression, renewed grounding, and retries.

trained on $\mathcal{D}_{\text{prim}}$ without full-task demonstrations from $\mathcal{T}_{\text{eval}}$; the agent generates object selection and task sequencing at inference.

A. Semantic Workflow and Execution Feedback

Given instruction u and context c , comprising the current scene information and retained task context, the agent generates a workflow:

$$W = \mathcal{P}(u, c) = (w_1, w_2, \dots, w_K),$$

where each step w_k specifies an object–destination assignment (o_k, d_k) . Here, o_k identifies the object selected under the task constraints, and d_k identifies its intended destination. Their current image regions are supplied by grounding.

Each active step retains its semantic assignment as grounding updates its regions. The resulting observation and execution report guide workflow progression or renewed grounding and execution.

a) *External Executable Skills:* ACE exposes two robot-facing skills: `MaskInterface` grounds the active sub-goal, and `PickPlace` executes the object-to-destination transfer. The agent framework manages workflow decomposition, memory queries, and sub-goal updates.

b) *Workflow Example:* Figure 2 illustrates an instruction to form the number 14. The agent selects $9 + 5$ and assigns cube 9 to paper A, the addition cube to B, and cube 5 to C. For each transfer, grounding produces the combined pick-and-place mask, the controller executes, and the agent assesses the resulting scene to advance or retry.

B. Mask-Mediated Vision-Action Interface

For each active workflow step w_k , `MaskInterface` generates a unified pick-and-place mask:

$$M_k = \mathcal{G}(w_k, I_t, m_t),$$

where I_t is the current image, m_t denotes the relevant memory state, and $M_k \in \{0, 127, 255\}^{H \times W}$ is a unified 8-bit grayscale mask. Pixel values encode the pick target, place target, and background:

$$M_k(x, y) = \begin{cases} 127, & \text{pick target,} \\ 255, & \text{place target,} \\ 0, & \text{otherwise.} \end{cases}$$

Given M_k , ACE initializes a tracker $T_k = \mathcal{T}_{\text{init}}(I_t, M_k)$ and updates it on subsequent frames to obtain $(T_k, M_{k,t+1}) = \mathcal{T}_{\text{update}}(T_k, I_{t+1})$. The tracked mask supplies the VA policy with updated target regions throughout the transfer, while the pixel labels retain their pick and place roles.

a) *Roles and object identity:* The two nonzero labels are scoped to the active transfer. A value of 127 can designate cube 9 in one step and the addition cube in the next; the assignment (o_k, d_k) retains their semantic identities upstream. The mask communicates their current regions and pick/place roles to the controller. Color and printed content enter object selection through the agent and grounding interface, while the controller receives the resulting spatial assignment. This

separation allows an instruction to change which object is manipulated without changing the policy’s input convention.

C. Reusable Pick-and-Place Control

PickPlace executes the active sub-goal using masks from MaskInterface. The VA policy conditions on two consecutive observations and returns an action chunk:

$$\hat{a}_{t:t+L-1} = \pi_{VA}(q_{t-1:t}, M_{k,t-1:t}),$$

where q_t is the robot proprioceptive state. DP is configured with a prediction horizon of 64 and $L = 32$ action steps per policy call. ACE executes these actions in a receding-horizon manner while updating observations and masks. The controller receives masks and proprioception; upstream modules process instructions and scene images. The same policy is used to transfer number cubes, operator cubes, and retrieval targets. After each attempt, the skill returns an execution report to the workflow manager.

D. Execution State and Persistent Context

Planning, tracking, and physical execution proceed on different timescales. ACE coordinates them through a shared scene-state manager with four memory components:

- **Live execution memory** maintains the current workflow position, active masks, tracker state, and execution progress on fast timescales.
- **Task-scoped semantic-visual memory** stores object names, aliases, roles, and visual associations for subsequent grounding and workflow decisions.
- **Appearance reference memory** stores image features or crops associated with known objects as references for later grounding calls.
- **Conversational context memory** maintains a bounded agent-call history containing the initial instruction and previous reasoning and tool invocations.

The live state follows tracking and control, while persistent object and conversational context remains available across agent calls. This distinction matters when manipulation changes visible object cues. After a cube rotates, its printed pattern may be hidden; earlier object associations and task context remain available to the agent alongside the current image when it assesses completion.

E. Sub-goal Completion and Retries

The execution cycle connects the active assignment to the next workflow decision:

- 1) **Ground the active assignment.** The agent invokes MaskInterface to locate o_k and d_k in the current scene and initialize their tracked mask.
- 2) **Execute the transfer.** PickPlace uses the mask-conditioned policy while tracking updates the target regions. The skill then returns an execution report.
- 3) **Assess the sub-goal.** The agent considers the resulting observation and report together with the active assignment and available context. A completed sub-goal advances the workflow; otherwise, the agent regrounds and retries.

The trial terminates when the agent determines three times that the same active sub-goal has not been completed. The count is attached to that sub-goal. Retries remain within the original trial. GA scores the final localization outcome of each required target, as defined in Section IV-C.

The memory-free variant retains the execution cycle, active workflow position, and tracker state while removing persistent object and conversational context (Appendix B-B).

IV. EXPERIMENTS

We evaluate whether a controller trained on approximately one hour of generic pick-and-place demonstrations can execute semantic tasks without full-task demonstrations of either evaluation task. The main results assess this task-level zero-shot composition capability. Two matched ablations then examine the controller’s visual input and the agent’s persistent context, measuring their contributions within the deployed framework.

A. Physical Setup and Protocol

We deploy the agentic reasoning engine on a cloud server with four NVIDIA RTX 3090 GPUs. The mask interface and VA policy run locally on a single RTX 3090. Each setting in Tables I and II is evaluated over 20 independent physical trials. We randomize initial object layouts and paraphrase instructions while preserving their semantic meaning. Implementation details are provided in Appendix B.

All trials in Tables I and II run autonomously after the initial instruction. Any grounding updates and subsequent execution attempts are handled by the evaluated system. For agent-controlled trials, the trial terminates when the agent determines three times that the same active sub-goal has not been completed.

B. Tasks

We evaluate ACE on two multi-step tabletop tasks requiring instruction-dependent object selection and execution. These tasks differ in semantic constraints, object selection logic, and workflow structure while sharing the same reusable pick-and-place primitive: **Semantic Formula Assembly**: The system selects number and operator cubes to form an expression satisfying the requested arithmetic condition. In all trials, three visible paper regions labeled A, B, and C specify the destinations for the first operand, operator, and second operand, respectively. **Constraint Retrieval**: The system must evaluate logical or numerical constraints to identify specific target objects and transport them to a designated spatial region. Examples are shown in Appendix A.

C. Evaluation Metrics

We report task Success Rate (SR), Grounding Accuracy (GA), and a Fine-Grained Score (FGS) for Semantic Formula Assembly. SR measures complete task success. GA measures the fraction of required targets that are correctly localized in the final grounding result, counting pick and place separately:

$$GA (\%) = 100 \frac{N_{\text{correct}}}{N_{\text{targets}}},$$

TABLE I

MAIN RESULTS ON MULTI-STEP TABLETOP TASKS. EACH METHOD–TASK SETTING USES 20 PHYSICAL TRIALS. SR DENOTES TASK SUCCESS AND FGS THE FORMULA-ASSEMBLY SCORE (MAXIMUM 20). GA MEASURES FINAL TARGET-LOCALIZATION ACCURACY: PICK AND PLACE ARE COUNTED SEPARATELY, AND INTERMEDIATE RETRIES ADD NO EXTRA ITEMS.

Method	Semantic Formula Assembly			Constraint Retrieval	
	avg. FGS (Max 20)	SR (%)	GA (%)	SR (%)	GA (%)
ACT (Imitation)	3.2	0.0	–	0.0	–
$\pi_{0.5}$ (End-to-End VLA)	3.0	0.0	–	0.0	–
ACE (w/o persistent memory)	16.8	55.0	96.7	70.0	90.0
ACE (Full Framework)	18.0	70.0	100.0	80.0	97.5

where N_{targets} is the total number of required targets and N_{correct} is the number correctly localized in the final result. Each required target is counted once; intermediate attempts and retries do not add items. FGS measures partial completion through semantic selection, grasping, placement, and final equation correctness, with a maximum of 20 points. The scoring rules are provided in Appendix B-A.

D. Baselines

We compare ACE with ACT [21] and $\pi_{0.5}$ [22], trained or fine-tuned with approximately one hour of full-task demonstrations collected using LeRobot [23]. These demonstrations include complete formula-assembly and constraint-retrieval trajectories. ACT is trained separately for each evaluation task. ACE’s VA policy uses approximately one hour of generic mask-conditioned pick-and-place demonstrations. This protocol evaluates whether a primitive-trained controller combined with task reasoning can execute the semantic tasks alongside models trained on complete task trajectories.

We also train a separate standalone Diffusion Policy for each task using the corresponding full-task demonstrations used for ACT (Table II(b)).

V. RESULTS

Table I reports task completion, partial progress, and target grounding. ACE completes 14 of 20 Semantic Formula Assembly trials (70.0% SR), with an average FGS of 18.0 out of 20, and 16 of 20 Constraint Retrieval trials (80.0% SR). Formula success requires a semantically valid expression and correct placement of its operands and operator; retrieval success requires selecting and placing objects that satisfy the instruction. Both are executed with the same controller trained on generic transfers, with complete evaluation-task trajectories absent from its training data.

ACT and $\pi_{0.5}$ complete no full tasks under the training configurations in Section IV, with formula-assembly FGS values of 3.2 and 3.0, respectively. The standalone DP comparison in Table II(b) also records no complete tasks and an FGS of 3.4.

ACE achieves 100.0% GA on Semantic Formula Assembly and 97.5% on Constraint Retrieval, measuring final target-localization outcomes. The difference between 100.0%

GA and 70.0% task success in formula assembly shows why grounding and complete execution are evaluated separately. FGS further records progress through selection, grasping, placement, and equation formation.

VI. ABLATIONS AND CONTROLLER COMPARISONS

a) Persistent memory: In Table I, the full ACE framework achieves 70.0% SR and 18.0 FGS on Semantic Formula Assembly, compared with 55.0% SR and 16.8 FGS without persistent memory. On Constraint Retrieval, the corresponding success rates are 80.0% and 70.0%. The observed gains are therefore 15 and 10 percentage points, corresponding to three and two additional successful trials. Grounding accuracy also increases from 96.7% to 100.0% on Semantic Formula Assembly and from 90.0% to 97.5% on Constraint Retrieval. Both variants use the same DP controller. The ablation removes persistent conversational and object context while retaining the live workflow and tracking state (Appendix B-B).

b) Completion uncertainty after appearance changes: In observed memory-free trials, cube rotation could hide the identifying surface pattern, leaving the agent uncertain whether it had completed the task. This is an ambiguity in interpreting the post-action scene: the current appearance may no longer expose the cue used to identify the object earlier. Persistent memory retains those earlier object associations and task information for subsequent assessment. This qualitative observation illustrates a role for context alongside the success-rate differences above.

c) Visual-input ablation: The two DP configurations in Table II(a) use identical training and execution settings and are evaluated on the same objects over 20 trials each. Mask-only DP succeeds in 18 of 20 trials (90.0%), compared with 6 of 20 (30.0%) for Mask + RGB, an observed difference of 60 percentage points. This comparison supports the mask-only visual interface under the evaluated conditions. Unseen objects in the evaluation have out-of-distribution colors and surface patterns.

The grasping comparison evaluates the controller’s spatial conditioning under the common training-data regime. The memory ablation evaluates context reuse during complete semantic tasks while holding the controller fixed. These

TABLE II

VISUAL-INPUT ABLATION AND CONTROLLER COMPARISONS. EACH SETTING USES 20 PHYSICAL TRIALS. TOP: SINGLE-CUBE GRASP SUCCESS. THE TWO DP INPUT CONFIGURATIONS USE THE SAME TEST OBJECTS AND MATCHED TRAINING AND EXECUTION SETTINGS. BOTTOM: TASK PERFORMANCE USING FULL-TASK DEMONSTRATIONS FOR STANDALONE DP AND PRIMITIVE-LEVEL DEMONSTRATIONS FOR ACE.

(a) Single-Cube Grasp Results					
Method	Setting	Grasp SR (%)			
$\pi_{0.5}$ LoRA	Seen	60.0			
$\pi_{0.5}$ LoRA	Seen + Unseen	40.0			
DP (Mask + RGB)	Shared test set	30.0			
ACE / DP (Mask Only)	Shared test set	90.0			

(b) End-to-End Task Comparison					
Method	Semantic Formula Assembly			Constraint Retrieval	
	avg. FGS (Max 20)	SR (%)	GA (%)	SR (%)	GA (%)
Standalone DP	3.4	0.0	—	0.0	—
ACE (Full Framework)	18.0	70.0	100.0	80.0	97.5

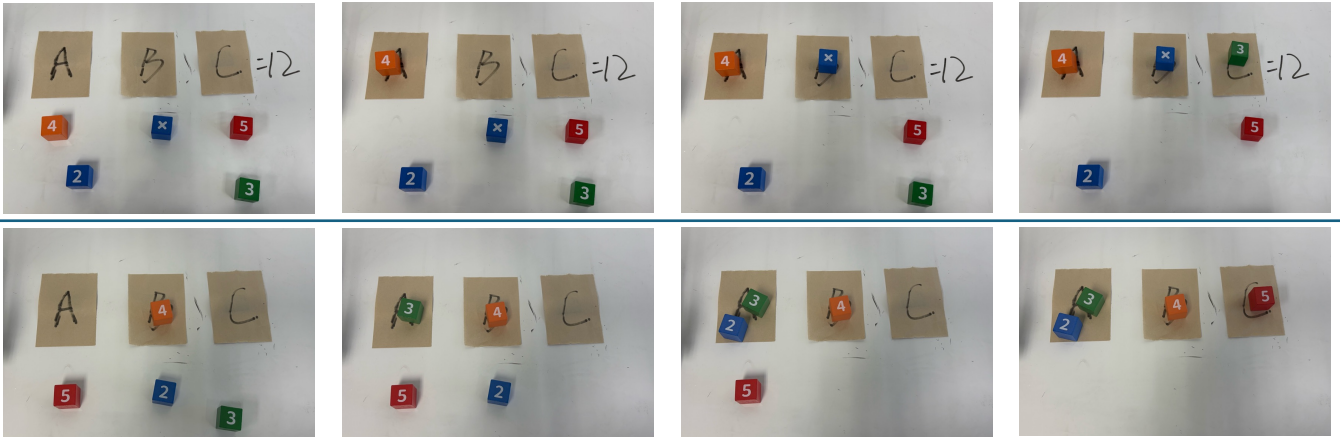


Fig. 3. **Execution sequences for the evaluation tasks.** **Top:** Semantic Formula Assembly arranges cubes into a valid expression. **Bottom:** Constraint Retrieval selects and places cubes according to the requested conditions (e.g., values < 4 to the left and > 4 to the right).

component comparisons support the design choices used in the main task-level zero-shot evaluation.

The $\pi_{0.5}$ LoRA controller, fine-tuned on 300 grasp demonstrations, achieves 60.0% grasp success on seen objects and 40.0% on the mixed seen/unseen setting. The standalone DP results in Table II(b) follow the full-task setup in Section IV.

VII. LIMITATIONS AND FUTURE WORK

The evaluation covers two cube-manipulation tasks and one pick-and-place primitive. Persistent object and conversational context are ablated jointly. Recovery observations are qualitative; verification accuracy, retry success, and execution time are not separately measured.

ACE’s design offers a route to broader task goals and scenes. The agent maps semantic requirements to object–destination assignments, while tracked masks provide a

shared spatial interface for executing them. This separation could support new selection rules and placement goals within the controller’s manipulation capabilities. Future work will evaluate transfer across diverse objects, scene layouts, and task constraints, measure grounding and execution robustness, and add learned primitives beyond pick-and-place.

VIII. CONCLUSION

ACE composes semantic tasks through object–destination assignments, tracked pick/place masks, and a reusable learned policy. Persistent context supplies earlier object and task information for outcome assessment as manipulation changes the scene. Trained only on generic transfers and without full-task demonstrations, the controller enables ACE to achieve 70% success on Semantic Formula Assembly and 80% on Constraint Retrieval.

APPENDIX A TASK DESCRIPTIONS

Figure 3 shows representative executions of the two evaluation tasks.

a) Semantic Formula Assembly: For a requested value y , let o_a and o_b denote selected number cubes and o_o the selected operator cube. A valid selection satisfies

$$\text{value}(o_a) \circ \text{value}(o_b) = y.$$

The corresponding transfers can be written as

$$W_{\text{formula}} = ((o_a, A), (o_o, B), (o_b, C)).$$

Grounding supplies the current regions for these role-specific transfers. The task combines semantic correctness of the expression with physical placement in the requested order.

b) Constraint Retrieval: Let $\mathcal{O}$ denote the objects under consideration and $\phi_u(o)$ the selection condition expressed by instruction u . The selected set is $\mathcal{O}_u = \{o \in \mathcal{O} : \phi_u(o) = 1\}$. Each selected object is assigned a destination $d_u(o)$ according to the instruction, producing transfers $(o, d_u(o))$. For the example in Fig. 3, a cube with value below 4 is assigned to the left region, and one with value above 4 to the right. The agent generates the sequence of transfers; all use the same mask encoding and learned pick-and-place controller as formula assembly.

APPENDIX B IMPLEMENTATION DETAILS

The implementation combines an agent, visual grounding and tracking, and a robot controller.

Planner and Workflow Manager: We implement the core language agent using the Qwen-Agent framework [24], powered by Qwen3.8-27B [25]. Qwen processes user instructions and rendered scene context to generate workflow steps and select subsequent skill calls. The two robot-facing skills, `MaskInterface` and `PickPlace`, are registered as callable tools within Qwen-Agent. The framework also supports state queries and memory management.

Mask-Mediated Vision-Action Interface: For the mask-mediated grounding ($\mathcal{G}$) and persistent tracking ($\mathcal{T}$), we utilize SAM3 [26] to generate pick-and-place masks and use Cutie [27] to update the masks over subsequent frames.

Robot Control: We use a SO-101 arm [28] with LeRobot [23] Diffusion Policy [29], trained on approximately one hour of generic object-to-region transfers specified by pick/place masks.

DP Training Configuration: The grayscale mask is replicated across three channels to form a $3 \times 480 \times 640$ policy input. The policy also receives a six-dimensional robot state and outputs six-dimensional actions. Its ResNet-18 visual backbone uses ImageNet-pretrained weights and 32 spatial-softmax keypoints. The diffusion denoiser uses channel widths [512, 1024, 2048], kernel size 5, eight groups, and a 128-dimensional diffusion-step embedding. DDPM uses 100 training diffusion timesteps, the `squaredcos_cap_v2` beta schedule, and noise prediction. The configured training

budget is 200,000 updates with batch size 16 and seed 1000. Adam uses learning rate 10^{-4} , weight decay 10^{-6} , $\beta = (0.95, 0.999)$, $\epsilon = 10^{-8}$, and gradient-norm clipping at 10. A cosine learning-rate schedule follows 500 warmup updates. Visual inputs use mean-standard-deviation normalization; states and actions use min-max normalization. Dataset image transforms are disabled.

A. Fine-grained Scoring

The Fine-Grained Score (FGS) measures partial completion in Semantic Formula Assembly.

Each episode consists of three sequential pick-and-place steps (first operand, operator, second operand). Each step is assigned a maximum score of 6 points:

- **2 points** for selecting a semantically correct cube. For policies without explicit object-identity outputs, selection is inferred from the cube toward which the robot directs its grasp attempt. A failed grasp can receive selection credit only if the attempted target is semantically correct;
- **2 points** for successfully grasping the selected cube (Physical execution);
- **2 points** for accurately placing the block into the target slot (Placement accuracy).

We additionally assign 2 points for final equation correctness, which evaluates whether the assembled expression mathematically matches the target value. The maximum score per episode is:

$$S_{\text{max}} = 3 \times (2 + 2 + 2) + 2 = 20.$$

a) Grounding counts and task outcomes: GA counts each required pick or place target once using its final localization outcome. A pair with a correct pick target and an incorrect place target contributes one correct target out of two. A target corrected during retries contributes one correct item; intermediate attempts add no items.

Correct grounding can coexist with failed execution. SR counts completed trials including retries; FGS records partial progress.

B. Ablation and Comparison Settings

The following settings define the memory ablation and additional controller and system comparisons:

- **$\pi_{0.5}$ LoRA single-cube grasping:** We fine-tune $\pi_{0.5}$ with LoRA on 300 single-cube grasp demonstrations and evaluate grasp success on seen and mixed seen/unseen object settings, with 20 physical trials per setting. Unseen refers to cubes with out-of-distribution colors and surface patterns.
- **DP visual-input ablation:** Mask + RGB receives RGB observations alongside the pick-and-place masks provided by ACE; Mask Only uses the masks as its visual input. The two configurations use identical training and execution settings and the same test objects, with 20 physical trials per configuration.

- **Standalone DP:** We train a separate standalone Diffusion Policy for each evaluation task on the corresponding full-task demonstrations used for ACT.
- **ACE (w/o persistent memory):** We retain the same DP controller and disable access to the persistent memory described in Section III-D: LLM conversation history, object names and aliases, textual object facts, visual identity features, and Code-as-Policies session history. Object memory is no longer injected into subsequent planning or visual prompts, and the `record_task_object_knowledge` tool is removed. The current workflow, step index, bounding boxes, and SAM/Cutie tracking state are retained to support ongoing task execution.

ACKNOWLEDGMENT

ChatGPT and Codex (OpenAI) assisted with result analysis (Sections V–VI), manuscript language polishing, schematic figure generation (Figs. 1–2), and writing portions of the code.

REFERENCES

- [1] M. Shridhar, L. Manuelli, and D. Fox, “CLIPort: What and where pathways for robotic manipulation,” in *Proceedings of the Conference on Robot Learning*, 2022.
- [2] M. Ahn *et al.*, “Do as i can, not as i say: Grounding language in robotic affordances,” *arXiv preprint arXiv:2204.01691*, 2022.
- [3] B. Zitkovich *et al.*, “Rt-2: Vision-language-action models transfer web knowledge to robotic control,” in *Conference on Robot Learning*. PMLR, 2023, pp. 2165–2183.
- [4] A. Zeng *et al.*, “Learning synergies between pushing and grasping with self-supervised deep reinforcement learning,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2018.
- [5] A. Brohan *et al.*, “RT-1: Robotics transformer for real-world control at scale,” in *Proceedings of Robotics: Science and Systems*, 2023.
- [6] J. Liang *et al.*, “Code as policies: Language model programs for embodied control,” in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2023.
- [7] W. Huang *et al.*, “Inner monologue: Embodied reasoning through planning with language models,” in *Proceedings of the 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, K. Liu, D. Kulic, and J. Ichnowski, Eds., vol. 205. PMLR, 14–18 Dec 2023, pp. 1769–1782. [Online]. Available: <https://proceedings.mlr.press/v205/huang23c.html>
- [8] Y. Ma, Z. Song, Y. Zhuang, J. Hao, and I. King, “A survey on vision-language-action models for embodied ai,” *arXiv preprint arXiv:2405.14093*, 2024. [Online]. Available: <https://arxiv.org/abs/2405.14093v1>
- [9] D. Driess *et al.*, “PaLM-E: An embodied multimodal language model,” in *Proceedings of the 40th International Conference on Machine Learning*, 2023, pp. 8469–8488.
- [10] S. Yao *et al.*, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations*, 2023. [Online]. Available: <https://mlanthology.org/iclr/2023/yao2023iclr-react/>
- [11] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian *et al.*, “ToolLLM: Facilitating large language models to master 16000+ real-world apis,” in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 9695–9717.
- [12] B. A. Plummer, L. Wang, C. M. Cervantes, J. C. Caicedo, J. Hockemaier, and S. Lazebnik, “Flickr30k entities: Collecting region-to-phrase correspondences for richer image-to-sentence models,” in *International Conference on Computer Vision*, 2015.
- [13] A. Kirillov *et al.*, “Segment anything,” in *International Conference on Computer Vision*, 2023.
- [14] H. K. Cheng, S. W. Oh, B. Price, A. Schwing, and J.-Y. Lee, “Tracking anything with decoupled video segmentation,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2023, pp. 1316–1326.
- [15] A. Stone *et al.*, “Open-world object manipulation using pre-trained vision-language models,” in *Proceedings of the 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229. PMLR, 2023, pp. 3397–3417. [Online]. Available: <https://proceedings.mlr.press/v229/stone23a.html>
- [16] Y. Zhong *et al.*, “DexGraspVLA: A vision-language-action framework towards general dexterous grasping,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, 2026, pp. 18 836–18 844. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/38953>
- [17] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, “VoxPoser: Composable 3d value maps for robotic manipulation with language models,” in *Proceedings of The 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, J. Tan, M. Toussaint, and K. Darvish, Eds., vol. 229. PMLR, 06–09 Nov 2023, pp. 540–562. [Online]. Available: <https://proceedings.mlr.press/v229/huang23b.html>
- [18] J. S. Park, J. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, ser. UIST ’23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: <https://doi.org/10.1145/3586183.3606763>
- [19] G. Wang *et al.*, “Voyager: An open-ended embodied agent with large language models,” *arXiv preprint arXiv:2305.16291*, 2023.
- [20] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 8634–8652.
- [21] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware,” in *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea, July 2023.
- [22] K. Black *et al.*, “ $\pi_{0.5}$: a vision-language-action model with open-world generalization,” in *9th Annual Conference on Robot Learning*, 2025.
- [23] R. Cadene *et al.*, “LeRobot: An open-source library for end-to-end robot learning,” in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: <https://openreview.net/forum?id=CiZMMAFQR3>
- [24] Qwen Team, “Generalizing an llm from 8k to 1m context using qwen-agent,” May 2024. [Online]. Available: <https://qwenlm.github.io/blog/qwen-agent-2405/>
- [25] —, “Qwen3.8-27B,” August 2026. [Online]. Available: <https://huggingface.co/Qwen/Qwen3.8-27B>
- [26] N. Carion *et al.*, “SAM 3: Segment anything with concepts,” in *International Conference on Learning Representations*, 2026.
- [27] H. K. Cheng, S. W. Oh, B. Price, J.-Y. Lee, and A. Schwing, “Putting the object back into video object segmentation,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2024, pp. 3151–3161.
- [28] Huggingface, “SO-101,” accessed: 2026-05-26. [Online]. Available: <http://huggingface.co/docs/lerobot/so101>
- [29] C. Chi *et al.*, “Diffusion policy: Visuomotor policy learning via action diffusion,” *The International Journal of Robotics Research*, vol. 44, no. 10-11, pp. 1684–1704, 2025.